Admission Control in Kubernetes

Role, Lifecycle, and Security Enforcement Beyond RBAC

Why Security Matters in Kubernetes

Kubernetes orchestrates hundreds or thousands of containers across clusters, making security paramount. Without proper controls, a single misconfigured pod could compromise an entire cluster or expose sensitive data.

Security in Kubernetes operates on a layered defence model, where each layer provides specific protection. Think of it like airport security: authentication is your ID check, RBAC is your boarding pass, and admission control is the metal detector that examines what you're carrying.

These layers work together to ensure that only authorised, validated, and secure requests reach your cluster's persistent storage.

Understanding RBAC

Role-Based Access Control

RBAC determines **who can do what** in your cluster. It answers the question: "Does this user have permission to create, read, update, or delete this resource?"

For example, a developer might have permission to create pods in their namespace but cannot access secrets or delete nodes.

Key benefits:

- Least privilege principle enforcement
- Scalable permission management
- Clear separation of duties

Made with GAMMA

Where RBAC Falls Short

RBAC is powerful but has clear limitations. It can verify permissions, but it cannot inspect the actual content of requests or enforce specific security policies.

No Image Control

Cannot restrict which container images can be used or verify image sources

Privileged Containers

Cannot prevent containers running with elevated privileges or host access

Metadata Enforcement

Cannot enforce mandatory labels, annotations, or resource limits

> "RBAC is like a bouncer checking your ID—it confirms you're allowed in, but can't inspect what you're carrying."

Made with GAMMA

Introducing Admission Control

Admission control acts as the **final checkpoint** before any API request is accepted into your cluster. It sits between authentication/authorisation and persistent storage, giving you the opportunity to validate or modify requests.

Think of it as airport security screening: after you've proven who you are (authentication) and shown your ticket (RBAC), security still needs to inspect your bag (admission control) before you board.

This layer can either **validate** requests (allow or reject) or **mutate** them (modify before acceptance), providing fine-grained control over what actually gets stored in etcd.

RBAC vs Admission Control

RBAC: WHO

- Authentication: Who are you?
- Authorisation: Do you have permission?
- Granular: User/Group → Verb (get, create, delete)
- Resource-based: Namespaces, pods, services

Admission Control: WHAT & HOW

- Validation: Does this request meet security policies?
- Modification: Should we enhance this request?
- Content-aware: Inspects actual resource specifications
- Policy-based: Images, privileges, limits, labels

They're complementary, not competing. RBAC ensures you're allowed to make the request; admission control ensures the request itself is safe and compliant.

Made with GAMMA

Kubernetes Request Lifecycle

Understanding the complete request flow helps clarify where each security layer operates.

User Request

kubectl or API client sends request

Authentication

Verify user identity (tokens, certificates, etc.)

Authorisation (RBAC)

Check permissions: can this user perform this action?

Admission Control

Validate and mutate request content before persistence

etcd Storage

Request accepted and stored in cluster state

Key point: Admission control runs after RBAC but before the request is persisted to etcd, making it the last opportunity to reject or modify requests.

Made with GAMMA

Deep Dive: How Admission Control Works

01

Request Interception

API server intercepts the request before it reaches etcd storage

02

Validation Phase

Validating admission controllers check if the request meets defined policies and security requirements

03

Mutation Phase

Mutating controllers can modify the request (add defaults, inject sidecars, set labels)

04

Final Decision

If any validator rejects the request, it's denied. Otherwise, the modified request proceeds to storage

Why is this needed after RBAC? Because RBAC only answers "Are you allowed?" while admission control answers "Is this safe and compliant?" A user might have permission to create pods, but admission control can still reject pods that use untrusted images or request excessive resources.

Made with GAMMA

Types of Admission Controllers

Mutating

These controllers **modify** requests before they're stored. They run first, allowing modifications that subsequent validators can then check.

Examples:

- PodPreset: Inject environment variables
- DefaultStorageClass: Set default storage
- SidecarInjector: Add sidecar containers
- NodeRestriction: Add node selectors

Like editing a document before final approval

Validating

These controllers **allow or reject** requests based on policies. They run after mutating controllers, validating the final form.

Examples:

- PodSecurity: Block privileged containers
- ImagePolicy: Restrict image registries
- ResourceQuota: Enforce limits
- ServiceAccount: Validate accounts

Like a final approval gate that can still reject

Enforcing Security Beyond RBAC

Admission control provides the fine-grained security policies that RBAC cannot. Here's where it makes the real difference:

Image Registry Control

Restrict containers to approved registries only. Prevent pulling images from untrusted sources, even if RBAC permits pod creation.

Security Policy Enforcement

Block privileged containers, prevent hostPath mounting, and enforce seccomp/apparmor profiles regardless of user permissions.

Resource Limits

Enforce CPU and memory limits on all pods, preventing resource exhaustion attacks even from authorised users with broad permissions.

Metadata Requirements

Mandate specific labels and annotations for compliance, auditing, or monitoring purposes—automatically rejected if missing.

> **Real scenario:** Developer has RBAC permission to create pods in production namespace. Admission control rejects pods using the latest:tag or requesting host privileges, protecting the cluster from risky deployments.

Made with GAMMA